

Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingeniería Informática

Clasificación Automática de
Afirmaciones Políticas
mediante Procesamiento del Lenguaje
Natural

Competencia Kaggle — Asignatura ABID

Tarea: Clasificación binaria de afirmaciones políticas
Dataset: 8.950 muestras de entrenamiento + 3.860 de test
Métrica: Macro F1 Score

Índice

1. Introducción	2
2. Descripción del problema	2
3. Descripción del conjunto de datos	2
3.1. Distribución de clases	3
3.2. Hallazgos relevantes por columna	3
3.3. Tratamiento de valores ausentes y alta cardinalidad	4
4. Metodología y tareas comunes	4
5. Análisis y preprocesamiento	5
5.1. Limpieza de texto	6
5.2. Representación textual	6
5.3. Variables categóricas y características derivadas	6
5.4. Problemas detectados y soluciones	6
6. Ingeniería de variables	7
6.1. Variables de texto	7
6.2. Variables de contexto	8
6.3. Variables de interacción	8
7. Modelos evaluados	8
7.1. Modelos lineales	8
7.2. Modelos <i>ensemble</i>	8
7.3. Transformers	9
8. Evaluación y resultados	10
9. Conclusiones	11

1. Introducción

La detección automática de afirmaciones falsas es una tarea cada vez más relevante dentro del Procesamiento del Lenguaje Natural (PLN), especialmente en contextos políticos, donde la desinformación puede influir de forma significativa en la opinión pública. Este proyecto aborda un problema de clasificación binaria: predecir si una afirmación política es verdadera o falsa a partir de su contenido textual e información contextual como el emisor, su afiliación política o el tema tratado.

Para resolver el problema se evaluaron distintos enfoques de aprendizaje automático, desde modelos clásicos hasta modelos más avanzados basados en *embeddings* semánticos, técnicas de *ensemble* y *boosting*.

Esta memoria resume el análisis del conjunto de datos, el preprocesamiento realizado, los modelos desarrollados, la metodología de evaluación y las principales conclusiones obtenidas.

2. Descripción del problema

El objetivo del proyecto consiste en desarrollar un sistema capaz de clasificar afirmaciones políticas como verdaderas o falsas utilizando técnicas de PLN y aprendizaje automático. Se trata de un problema de clasificación binaria, donde la variable objetivo toma el valor 1 para las afirmaciones verdaderas y 0 para las falsas.

El *input* principal es el texto de la afirmación, pero también se cuenta con metadatos como tema, hablante (*speaker*), ocupación, estado y afiliación política, que pueden aportar señal adicional al modelo.

El reto central es que la veracidad no siempre se puede inferir únicamente del texto. Dos afirmaciones pueden ser lingüísticamente similares y pertenecer a clases opuestas, de modo que ningún enfoque único resulta suficiente. A eso se suma el desequilibrio de clases presente en el *dataset*, por lo que se emplea el **Macro F1** como métrica principal, ya que permite evaluar el rendimiento de forma equilibrada entre ambas clases. Se probaron varios enfoques: desde modelos clásicos con representaciones TF-IDF hasta modelos más avanzados con *embeddings* semánticos y técnicas de *ensemble*, buscando el mejor *trade-off* entre precisión y generalización.

3. Descripción del conjunto de datos

El conjunto de datos utilizado [?] contiene afirmaciones políticas acompañadas de información contextual. Cada registro incluye las columnas descritas en la Tabla 1.

Cuadro 1: Descripción de las columnas del *dataset*.

Columna	Tipo	Descripción
id	Identificador	Solo para trazabilidad; no predictivo.
label	Target binario	0 = verdadero, 1 = falso.
statement	Texto	El texto de la afirmación política.
subject	Categórica	Tema(s) de la afirmación (hasta 4 por fila).
speaker	Categórica	Persona que hace la afirmación.
speaker_job	Categórica	Cargo/profesión del hablante (27,7 % ausentes).
state_info	Categórica	Estado geográfico (EE. UU.).
party_affiliation	Categórica	Partido político del hablante.

3.1. Distribución de clases

El *dataset* contiene **8.950 muestras** y presenta un desequilibrio moderado entre clases: aproximadamente el 65 % pertenecen a la clase 1 (falsa) y el 35 % a la clase 0 (verdadera). Los pesos calculados a partir de ese ratio son $w_0 = 1,42$ y $w_1 = 0,77$, y se aplicaron de forma consistente en todos los modelos. Este desequilibrio motivó el uso de Macro F1 como métrica de evaluación principal.

3.2. Hallazgos relevantes por columna

Texto de la afirmación (statement). Es la columna más predictiva. La mediana es de aproximadamente 20 tokens. Contiene señales lingüísticas características de las noticias falsas: lenguaje absolutista, negaciones, números específicos y palabras de cobertura (*hedge words*).

Tema (subject). Hay 115 temas únicos distribuidos según una ley de potencia; los 20 más frecuentes concentran el 79,5 % de los registros. La relación con la etiqueta es estadísticamente significativa ($\chi^2 = 95,87$, $p < 0,0001$). El 32 % de las filas tiene más de un tema, lo que requirió un tratamiento especial.

Hablante (speaker). Alta cardinalidad con cola larga. Los hablantes más frecuentes son Barack Obama, Donald Trump y Hillary Clinton. La tasa de veracidad histórica por hablante es el predictor más fuerte del *dataset*, aunque también el más propenso a introducir *leakage* si se calcula incorrectamente.

Cargo (speaker_job). 1.018 títulos únicos; el 84,6 % aparece menos de cinco veces. Hay 2.482 valores ausentes (27,7 %).

Afiliación política (`party_affiliation`). 22 valores únicos sin ausentes. Demócratas y Republicanos cubren el 77,8 % del *dataset*.

3.3. Tratamiento de valores ausentes y alta cardinalidad

Los valores nulos en `speaker_job` y `state_info` se sustituyeron por la categoría `unknown`, conservando todas las muestras disponibles. Las categorías poco frecuentes se agruparon bajo la etiqueta `other` (umbral de 5 apariciones para `speaker` y `speaker_job`; de 10 para `subject`), reduciendo el ruido sin perder la señal de infrecuencia. La columna `subject` se dividió en temas separados y se normalizó, generando además el número de temas asociados a cada afirmación.

4. Metodología y tareas comunes

El desarrollo del proyecto se llevó a cabo de forma colaborativa distribuyendo el trabajo entre los integrantes del grupo. Las decisiones sobre preprocesamiento, evaluación y comparación de resultados se tomaron conjuntamente para garantizar coherencia. La planificación se estructuró en las fases recogidas en la Tabla 2.

Cuadro 2: Calendario de trabajo.

Fechas	Actividad
24/04	Primera reunión: revisión del enunciado, <i>dataset</i> y reglas de entrega. Reparto de trabajo.
25/04	Revisión de datos y preprocesado inicial. Criterios comunes: limpieza de texto, nulos, categorías.
26/04	Línea base compartida: <i>baseline</i> común, misma métrica de evaluación, registro de errores.
27/04–30/04	Modelos independientes (fase 1): modelo lineal, árbol con variables tabulares, TF-IDF + <i>metadata</i> .
01/05–04/05	Tuning y mejora: hiperparámetros, variantes de preprocesado, selección de <i>features</i> .
05/05–08/05	Modelos independientes (fase 2): segunda familia o mejora clara del primer intento.
09/05–12/05	Análisis cruzado: comparación de modelos, elección de 2–3 enfoques, decisión sobre ensamblado.
13/05–15/05	Pipeline final: reentrenamiento, reproducibilidad, generación del CSV de test.
16/05–19/05	Cierre y entrega: últimas submissions, código final, memoria y presentación.

Para garantizar la reproducibilidad y el control de versiones se utilizó GitHub como repositorio común. El desarrollo y la experimentación se realizaron en Jupyter Notebooks con Python y librerías como `pandas`, `scikit-learn` [?], `XGBoost` [?], `LightGBM` [?], `CatBoost` [?] y `Hugging Face Transformers` [?]. La evaluación y la generación de *submissions* se llevaron a cabo mediante Kaggle.

Para la búsqueda de hiperparámetros se emplearon `RandomizedSearchCV` y `HalvingGridSearchCV` [?]. En los modelos Transformer también se ajustaron parámetros de entrenamiento profundo como *learning rate*, *batch size*, *dropout* y número de épocas.

5. Análisis y preprocesamiento

El preprocesamiento y la ingeniería de variables tuvieron un papel fundamental en el rendimiento de los modelos. El objetivo principal fue reducir el ruido, representar correctamente la información textual y contextual, y evitar problemas de sobreajuste o *leakage*.

5.1. Limpieza de texto

Para los modelos clásicos basados en TF-IDF [?] se aplicó un preprocesamiento más agresivo sobre `statement`: conversión a minúsculas, eliminación de puntuación, normalización de contracciones, reemplazo de números por un token genérico, tokenización, eliminación de *stopwords* [?] y lematización [?]. Se conservaron negaciones como *not* o *never* por su relevancia para la clasificación. Para los Transformers no se aplicó limpieza agresiva, ya que aprovechan mejor el lenguaje natural original.

5.2. Representación textual

La representación principal se realizó mediante TF-IDF con n -gramas de palabras y caracteres [?]. También se generaron *embeddings* semánticos con Sentence Transformers [?]: primero `all-MiniLM-L6-v2` (384 dimensiones) [?] y después `all-mpnet-base-v2` (768 dimensiones) [?], permitiendo capturar relaciones semánticas más complejas.

5.3. Variables categóricas y características derivadas

Además de la reducción de cardinalidad descrita en la Sección 3, se generaron características derivadas relacionadas con el contexto político: tasas históricas de veracidad asociadas al hablante, partido político o tema tratado. Para evitar *leakage*, estas estadísticas se calcularon mediante estrategias *out-of-fold* (OOF) dentro de la validación cruzada. La variable `subject` se codificó con `MultiLabelBinarizer`, generando variables binarias independientes para cada tema relevante.

5.4. Problemas detectados y soluciones

Durante el desarrollo se identificaron varios problemas críticos, resumidos en la Tabla 3.

Cuadro 3: Principales problemas detectados y sus soluciones.

Problema	Descripción	Impacto
Desequilibrio de clases	35 % verdadero vs. 65 % falso; clase 0 sistemáticamente subpredicada.	Alto
Alta cardinalidad	<code>speaker</code> (2.634), <code>speaker_job</code> (1.018): riesgo de sobreajuste.	Alto
Dominancia de <i>feature</i>	<code>fe_speaker_true_rate</code> con 6,7–8,76× la ganancia del siguiente.	Alto
<i>Leakage</i> de target	Tasas de veracidad calculadas sobre todo el <i>dataset</i> .	Crítico
Bug <code>statement</code> en OE	Texto crudo codificado como pseudo row-ID por <code>OrdinalEncoder</code> .	Crítico
<i>Stopwords</i> en TF-IDF	<code>max_df=0.9</code> dejaba pasar artículos como <i>top features</i> .	Medio
Umbral subóptimo	Threshold 0.5 sistemáticamente favorece la clase mayoritaria.	Alto
Overfitting del Transformer	8.950 muestras insuficientes para <i>fine-tuning</i> sin regularización adicional.	Medio

6. Ingeniería de variables

El pipeline de variables se diseñó para capturar señales desde tres ángulos distintos: el texto de la afirmación, las características del hablante y el contexto político, y las interacciones entre ambos.

6.1. Variables de texto

- **Vectoriales-semánticas:** *embeddings* de oraciones con `all-MiniLM-L6-v2` o `all-mpnet-base-v2`.
- **Léxicas y estructurales:** longitud en caracteres y palabras, ratio de mayúsculas, densidad de dígitos, errores ortográficos, conteos NER (PERSON, ORG, GPE, DATE, NUM con spaCy [?]).
- **Lingüísticas de detección de engaño:** conteo de negaciones (`fe_negation_count`), palabras de cobertura (`fe_hedge_count`), lenguaje absolutista (`fe_absolutist_count`), números específicos (`fe_numeral_count`), legibilidad Flesch-Kincaid (`fe_readability`) y sentimiento con TextBlob (`fe_sentiment_polarity`, `fe_sentiment_subjectivity`).

6.2. Variables de contexto

Para el hablante, tema, partido y cargo se generaron: frecuencia en el *dataset*, *flag* de rareza, longitud del nombre y si contiene prefijo de título. Las variables más predictivas fueron las **tasas OOF de veracidad** (`fe_speaker_true_rate`, `fe_subject_true_rate`, `fe_party_true_rate`, `fe_speaker_job_true_rate`), calculadas exclusivamente dentro del *fold* de entrenamiento para evitar *leakage*.

6.3. Variables de interacción

Se generaron claves compuestas de pares de columnas (`speaker × subject`, `speaker × party`, `subject × party`, etc.) codificadas con `OrdinalEncoder`. Los modelos de árbol explotan estas interacciones con una sola condición de *split*, equivalente a dos *splits* secuenciales en los modelos lineales.

7. Modelos evaluados

Con el objetivo de comparar distintos enfoques para la detección de afirmaciones falsas, se evaluaron modelos de diferente complejidad, desde algoritmos lineales clásicos hasta modelos *ensemble* y arquitecturas Transformer. Todos los modelos se entrenaron con validación cruzada estratificada de cinco *folds* más un *holdout* fijo del 20 %, no tocado hasta la evaluación final. Las métricas principales fueron Macro F1 y ROC-AUC.

7.1. Modelos lineales

Los primeros modelos evaluados fueron Regresión Logística [?] y Linear SVM, basados en representaciones TF-IDF con *n*-gramas de palabras y caracteres. La Regresión Logística destacó por su simplicidad e interpretabilidad, obteniendo resultados competitivos como *baseline*. La regularización fue $C \in [0, 1, 1, 0]$ con penalización L2 [?] o L1. El mejor Macro F1 lineal (0,611) se obtuvo con *nested CV* (búsqueda interna de hiperparámetros en tres *folds*), que evita el sesgo optimista en la selección de parámetros.

Aunque estos modelos ofrecieron *baselines* sólidos, presentan limitaciones para capturar relaciones semánticas complejas: el modelo no puede aprender que un hablante republicano hablando de economía durante una elección tiene más probabilidad de mentir que lo que predice cualquier *feature* por separado.

7.2. Modelos *ensemble*

Posteriormente se evaluaron Random Forest [?], XGBoost [?], LightGBM [?] y CatBoost [?].

Random Forest. Se usaron *embeddings* semánticos y variables derivadas del historial del hablante. Mostró una mejora significativa sobre los modelos lineales, especialmente gracias al cambio de TF-IDF a *embeddings* (+0,014 Macro F1). Se descubrió también un bug crítico: la columna `statement` cruda pasaba por el `OrdinalEncoder` como pseudo row-ID, haciendo que el árbol memorizara las filas de entrenamiento. La configuración óptima fue `min_samples_leaf=6` y `max_features=0.5`.

LightGBM. Reveló el problema de dominancia de *features*: `fe_speaker_true_rate` acaparaba 6,7× la ganancia del siguiente *feature*. La mejor configuración (opción B) consistió en eliminar dicha variable, lo que mejoró el *holdout* en 0,012 puntos de Macro F1 y 0,011 de ROC-AUC, confirmando que memorizaba identidades del *training set* en lugar de generalizar.

CatBoost. Ataca el problema de dominancia desde la raíz mediante *ordered boosting* [?]: las estadísticas de *target* se computan solo con las muestras anteriores en una permutación aleatoria, eliminando el leakage interno. Los árboles simétricos añaden regularización implícita. CatBoost+OptB (con `fe_speaker_true_rate` eliminada) fue el mejor modelo individual: Macro F1 = 0,629, ROC-AUC = 0,674, y el primer *threshold* inferior a 0,5 (0,46), evidenciando mejor calibración probabilística.

Stacking. Se implementó una estrategia de *stacked generalisation* [?] combinando cuatro modelos base (LR, RFC, LGBM, CatBoost) mediante un meta-clasificador Logistic Regression con $C = 0,1$. Las probabilidades OOF de cada modelo forman la matriz de meta-*features*. Random Forest recibió el coeficiente más alto del meta-clasificador (1,51–1,64) a pesar de un ROC-AUC individual menor, porque sus errores son los más decorrelacionados de los modelos *boosted*. Tras el upgrade del *embedding* de `all-MiniLM-L6-v2` (384 dim) a `all-mpnet-base-v2` (768 dim), el *stacking* alcanzó Macro F1 = 0,643 y ROC-AUC = 0,700.

7.3. Transformers

La última familia de modelos se basó en arquitecturas Transformer preentrenadas, concretamente DeBERTa-v3 [? ?]. Para incorporar la *metadata* contextual se adoptó la estrategia de **texto formateado**: el *input* al modelo tiene la forma:

```
"speaker: john edwards | party: democrat | subject: health-
care | {statement}"
```

Esto permite al mecanismo de atención cruzar información entre el texto de la afirmación y la identidad del hablante sin cambios de arquitectura, superando con creces

el MLP híbrido paralelo (que producía interferencia de gradientes y val macro_f1 fijo en 0,5836 en todas las épocas).

Se ajustaron hiperparámetros como *learning rate*, *batch size*, *dropout* (0,3 en la cabeza de clasificación), Layer-wise Learning Rate Decay (LLRD) [?] y número de épocas. El mejor *checkpoint* de cada corrida se seleccionó por val_macro_f1, no por val_loss.

Los mejores resultados no se obtuvieron con Transformers *standalone*, sino mediante **late fusion**: las predicciones OOF de DeBERTa-v3-base se incorporaron como quinto modelo base en el *stacking*. El meta-clasificador asignó al Transformer el coeficiente más alto (1,97), reconociendo que aporta señal de texto puro que los árboles no tienen. Esta combinación alcanzó el mejor resultado del proyecto.

8. Evaluación y resultados

La evaluación de los modelos se realizó con validación cruzada estratificada de cinco *folds* y un *holdout* fijo del 20 % reservado para la evaluación final. Debido al desequilibrio presente en el *dataset*, las métricas principales son Macro F1 y ROC-AUC.

La Tabla 4 muestra los resultados de los modelos más relevantes.

Cuadro 4: Resultados en *holdout* de los modelos evaluados.

Modelo	Macro F1	ROC-AUC	Uml
LR <i>baseline</i> (TF-IDF)	0,604	0,654	0,5
LR + <i>nested CV</i> + <i>embeddings</i>	0,611	0,660	—
RFC Paso 5 (<i>embeddings</i> + NER + <i>job_true_rate</i>)	0,621	0,667	0,5
LGBM Opción B (sin <i>speaker_true_rate</i>)	0,618	0,679	0,5
CatBoost + Opción B	0,629	0,674	0,4
<i>Stacking</i> Run 2 (mpnet 768 dim)	0,643	0,700	0,0
Transformer Exp. 3 — <i>Late Fusion</i> DeBERTa-small	0,644	0,709	—
Transformer Exp. 4b — DeBERTa-base + <i>Late Fusion</i>	0,647	0,711	—

A lo largo del proyecto se observó una mejora progresiva al incorporar representaciones textuales más avanzadas, variables contextuales y técnicas de combinación de modelos. Los saltos más significativos fueron:

- Cambio de TF-IDF a *embeddings* en RFC: +0,014 Macro F1.
- Eliminación de *fe_speaker_true_rate* en LGBM: +0,012 *holdout* F1 (confirmando memorización de identidades).
- Upgrade de *embedding* de 384 a 768 dimensiones en el *stacking*: +0,010 Macro F1.

- Incorporación del Transformer en *late fusion*: +0,005 Macro F1 y +0,009 ROC-AUC sobre el *stacking* puro.

Una de las observaciones más llamativas fue que algunas variables históricas, aunque muy predictivas, podían introducir sobreajuste si no se calculaban correctamente mediante estrategias OOF. Los Transformers mejoraron notablemente al incorporar *metadata* contextual directamente en el texto de entrada, pero los mejores resultados se obtuvieron siempre mediante estrategias híbridas de *late fusion* que combinaban las fortalezas de ambas familias de modelos.

9. Conclusiones

En este proyecto se abordó un problema de clasificación de afirmaciones políticas mediante técnicas de PLN y aprendizaje automático, evaluando enfoques que van desde modelos lineales clásicos hasta arquitecturas Transformer y estrategias *ensemble* avanzadas.

Los resultados muestran que la información contextual — hablante, tema y afiliación política — tuvo un impacto muy relevante en el rendimiento. La combinación de esta *metadata* con representaciones textuales avanzadas permitió mejorar significativamente las métricas de clasificación. A continuación se resumen las lecciones más importantes:

1. **El texto es necesario pero no suficiente.** El *feature* más predictivo en los modelos de árbol fue siempre la tasa de veracidad histórica del hablante, no los *embeddings*; pero esa misma variable era la que más sobreajustaba cuando se calculaba incorrectamente.
2. **Eliminar el *feature* dominante mejoró la generalización.** Quitar `fe_speaker_true_rate` mejoró el *holdout* en LGBM (+0,012 F1) y fue también aditivo en CatBoost, confirmando que el modelo memorizaba identidades del *training set*.
3. **El *stacking* explota complementariedad real.** Random Forest, pese a ser el modelo individual más simple, recibió el coeficiente más alto en el meta-clasificador por sus errores decorrelacionados de los modelos *boosted*.
4. **El texto formateado supera al MLP híbrido.** Inyectar *metadata* como tokens de texto es más efectivo que una rama paralela de red neuronal en *datasets* pequeños.
5. **Los Transformers ganan más en *late fusion* que *standalone*.** DeBERTa-base solo alcanzó 0,640 de Macro F1; en *late fusion* con el *stacking* llegó a 0,647.
6. **La calibración del umbral de decisión es crítica.** El umbral óptimo varió entre 0,46 (CatBoost, bien calibrado) y 0,62 (*stacking* sin `class_weight` en el meta-clasificador).

7. **Entender las fallas vale más que añadir complejidad.** El bug del pseudo row-ID, la dominancia de *features* y la interferencia de gradientes se resolvieron cada uno con cambios relativamente simples; resolverlos fue más valioso que agregar capas o variables adicionales.

El mejor resultado del proyecto — Macro F1 = **0,647** y ROC-AUC = **0,711** — se alcanzó con DeBERTa-v3-base en *late fusion* con *stacking* de cuatro modelos *ensemble*. Esto demuestra que combinar modelos con mecanismos de aprendizaje distintos permite aprovechar mejor las fortalezas de cada enfoque y mejorar la capacidad de generalización del sistema.